LABORATUVAR 7 · EFO MAKİNA · BESLEME BANDI & SARSAK

AI Safety Challenge

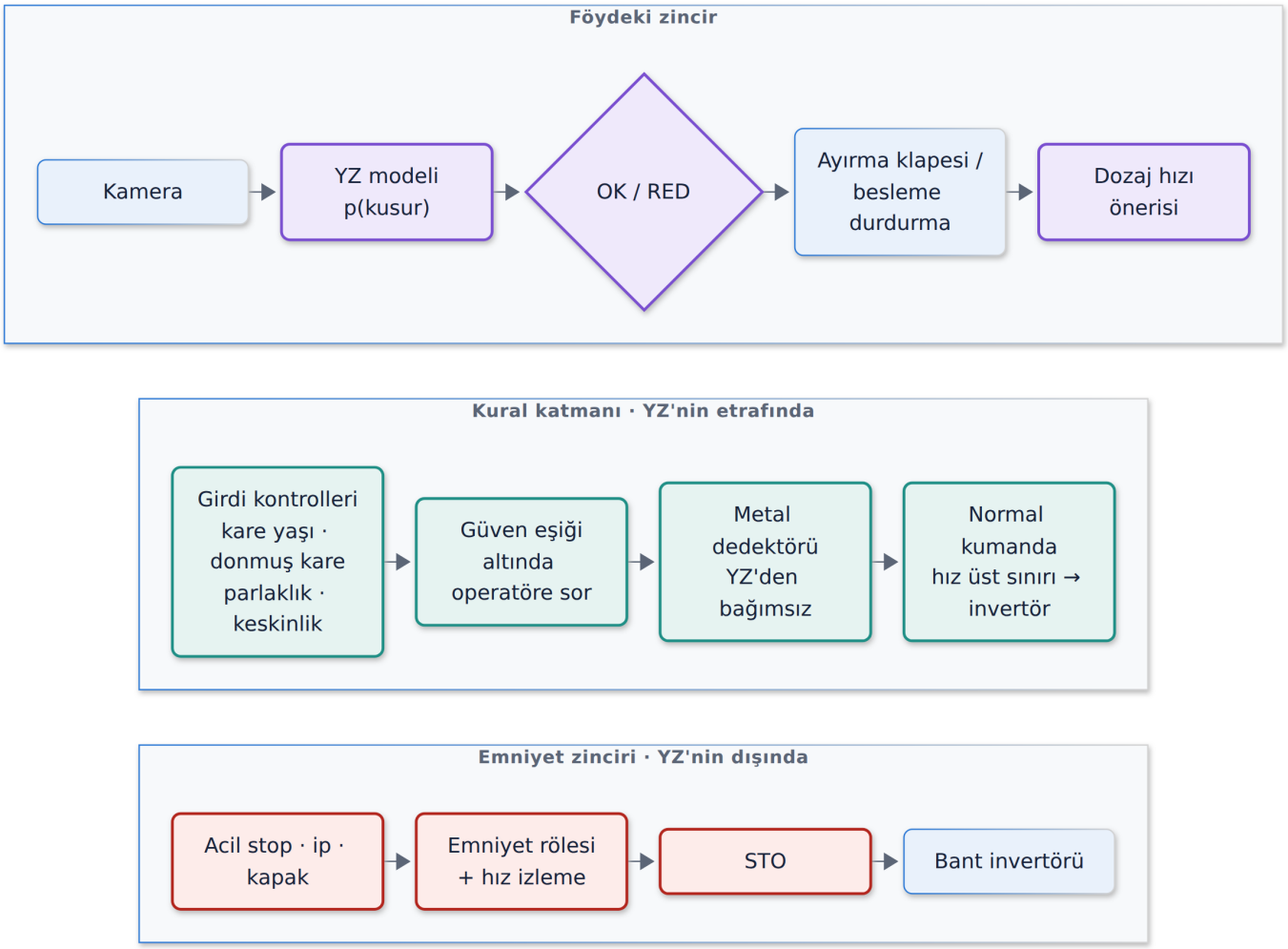
Çıktı: AI Fail-Safe Decision

AD SOYAD	GRUP	EVİDENCE PACK	MAKİNE
Yağız Enes Doğan	Gündüz - B	EP 03 + EP 15	Efo Makina · Besleme Bandı & Sarsak (Titreşimli Konveyör)

Bağımsız akademik çalışmadır; Efo Makina ile bağlantısı yoktur. Analizin birçok kısmı varsayıma dayanır ve gerçek ürünün emniyet durumu hakkında bir iddia değildir.

Kavram: Prediction ≠ Safety: Yapay zekâ bir karar üretir; güvenliği garanti eden mimari YZ'nin **dışında** tanımlanmalıdır. Araçlar: fail-safe · human override · monitoring · confidence threshold · fallback.

Efo bağlamı: Efo'nun ürünlerinde YZ yok. PaketPro sürümündeki gibi bir **proje aşaması** değerlendirmesi kuruldu (V20): besleme bandının üstündeki portala bir kamera eklenecek [Y2]. YZ, bantta yabancı cisim (metal, taş) ve aşırı boy parça arayacak. RED gelince parça ayrılacak ya da besleme duracak; OK gelince YZ dozaj hızını önerecek. Yabancı cisim yongalama bıçaklarına zarar verir [ÇIKARIM]. Modelin skorları sentetiktir; gerçek bir modelin performansı değildir.



Şekil 1 — Föydeki zincir (üstte), YZ'nin etrafındaki kural katmanı (ortada) ve YZ'den bağımsız emniyet zinciri (altta). YZ yalnız normal kumandaya öneri verir.

Temel karar: YZ bir emniyet fonksiyonu değildir. YZ bir üretim ve süreç fonksiyonudur: bıçağı ve ürün kalitesini korur. Hattaki insanı koruyan SF1–SF4 (LAB 1) YZ'den geçmez. Bu tercih bilinçli:

- Makine öğrenmesiyle davranışı gelişen ve emniyet fonksiyonu üstlenen bileşenler, 2023/1230 Ek I Kısım A'da üçüncü taraf uygunluk değerlendirmesi gereken kategoride [Y27] SEKTÖR-TİPİK.
- Böyle bir bileşen YZ Yasası'nda da yüksek riskli sayılır [Y28] SEKTÖR-TİPİK.
- YZ'yi emniyet zincirinin dışında tutmak, hem güvenceyi kolay doğrulanabilir bir kablolu zincirde bırakıyor hem de bu yükü doğurmuyor ÇIKARIM.

Görev — üç başarısızlıkta güvenli davranış

1 Model yanlış sınıflandırdı: Kusurlu parçaya OK verdi. Sistem ne yapmalı? İkinci kontrol / insan onayı var mı?

Yanlış OK'yi tek bir modelin doğruluğuna bırakmayız. Üç katman var:

1. **Güven eşiği:** OK ancak model yeterince eminse otomatik geçer; değilse operatöre sorulur (Q3).
2. **YZ'den bağımsız ikinci kanal:** Metal dedektörü, metal parçaları YZ'den ayrı bir fiziksel ilkeyle yakalar. Dedektör "metal" derse YZ'nin OK'si geçersizdir; **bağımsız kanal kazanır**. Taş ve aşırı boy parça için ikinci kanal, yongalama makinesinin aşırı akım koruması ve operatörün periyodik numune kontrolüdür VARSAYIM.
3. **RED her zaman kazanır:** Düşük güvenli bir RED bile parçayı ayırır. Yanlış ret yalnızca malzeme kaybıdır; yanlış OK ise bıçak hasarıdır. Fail-safe asimetri budur.

Sentetik kümede sonuç:

Düzen	Güven eşiği	Kaçak (kusurların)	Operatöre sorulan (tüm parçaların)	Yanlış ret	Hedef ≤ %1,0
Eşiksiz (her OK geçer)	0,50	%6,57	%0,00	%0,49	✖
Güven eşiği	0,61	%0,97	%2,18	%0,49	✔
Güven eşiği + metal dedektörü	0,56	%0,88	%0,78	%0,49	✔

Sentetik küme: 20000 parça, kusur oranı %2,0, kusurların %60'ı metal. Operatör kapasitesi tüm parçaların %5'i.

- **Eşiksiz YZ**, kusurların %6,57'sini geçiriyor; hedef %1.
- **Güven eşiği** (0,61) hedefi karşılıyor. Bedeli, parçaların %2,18'inin operatöre sorulması.
- **Metal dedektörü** eklenince eşik 0,56'ya iniyor ve operatör yükü %0,78'e düşüyor. Bağımsız ikinci kanal, insanın yükünü de azaltıyor.

Metal dedektörü hesabı, YZ ile dedektörün hatalarının **bağımsız** olduğunu varsayıyor. Bağımsızlık gerçek kurulumda kanıtlanmalı (V20).

Bağımsızlık bozulursa (7.4). İki kanal aynı nedenle birlikte yanılabilir: örneğin ıslak ve iri bir yonga yığını hem kameranın görüşünü kapatır hem de dedektörün sinyalini bastırır. Model, YZ'nin kaçırdığı metal parçaların ρ kadarını dedektörün de aynı nedenle kaçırdığını kabul ediyor VARSAYIM (V26):

Ortak neden payı ρ	Dedektörün kaçırma olasılığı	Kaçak (kusurların)	Hedef ≤ %1,0
0,00	%5,0	%0,88	✔
0,04	%8,8	%0,99	✔
0,08	%12,6	%1,10	✖
0,12	%16,4	%1,21	✖
0,16	%20,2	%1,32	✖
0,20	%24,0	%1,43	✖

Seçilen eşik 0,56 (YZ + metal dedektörü). Kaçak hedefi, ortak neden payı $\rho = 0{,}045$* değerini aştığında aşılıyor: YZ'nin kaçırdığı her 22 metal parçadan birini dedektörün de aynı nedenle kaçırması yetiyor.

Sonuç, ikinci kanalın değerinin neredeyse tamamen bağımsızlıktan geldiğini gösteriyor. Küçük bir ortak neden payı bile hedefi bozuyor (`test_ortak_neden_esigi_asilinca_kacak_hedefi_asilir`). Bu yüzden iki kanalın farklı fiziksel ilkeye dayanması ve ortak neden payının devreye almada ölçülmesi gerekiyor ÇIKARIM [Y29].

2 Girdi manipüle edildi: Kamera verisi bozuk/sahte. Nasıl fark edilir? Şüpheli girdide ne yapılır?

Nasıl fark edilir (kod: `girdi_supheli`):

- **Kare yaşı:** Görüntü 200 ms'den eskiyse yayın gecikmiş ya da kayıt tekrar oynatılıyor olabilir.
- **Donmuş kare:** Aynı görüntü (aynı hash) üst üste 5 kez geliyorsa kamera donmuş ya da araya sahte bir sabit görüntü sokulmuş olabilir.
- **Parlaklık ve keskinlik:** Lens kapalı, toz kaplı ya da kamera yönü değişmiş olabilir.
- **Bağımsız sensörle tutarsızlık:** YZ "OK" derken metal dedektörü "metal" diyorsa biri yanılıyor ya da kandırılıyor.

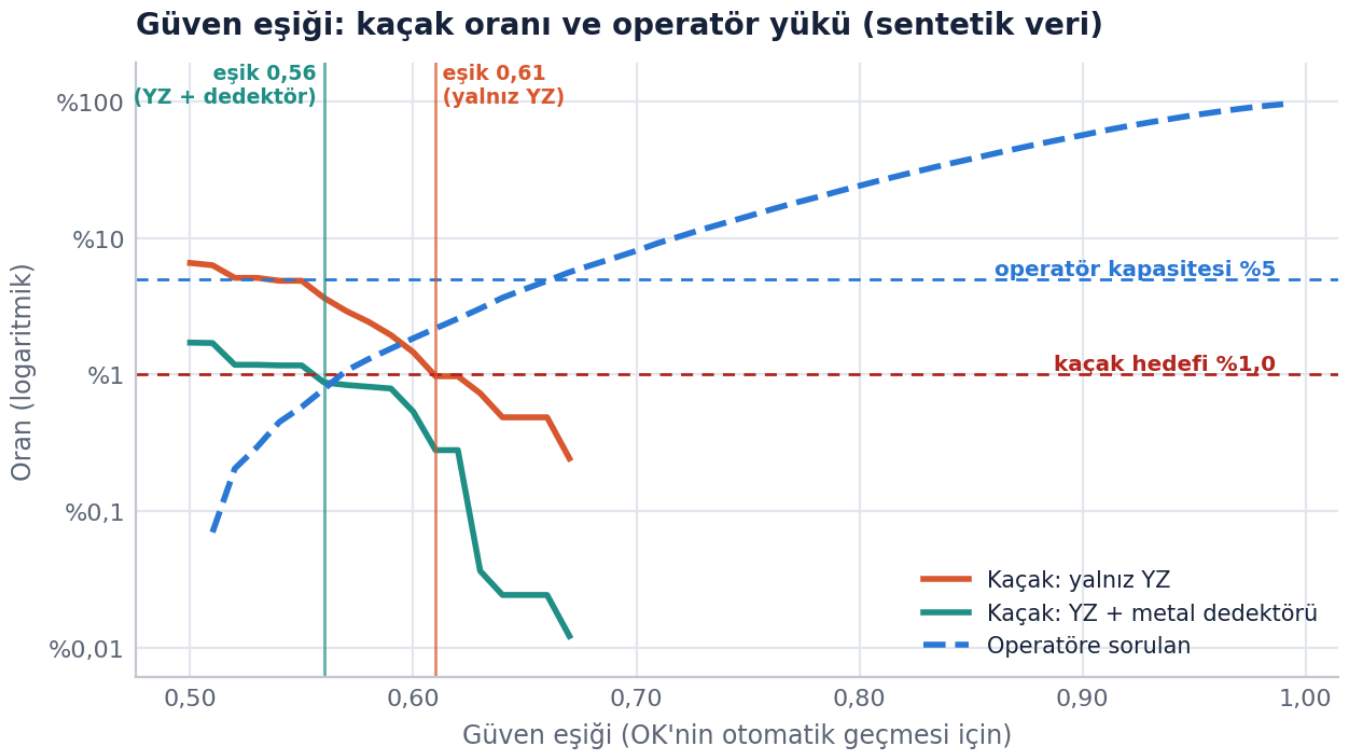
Şüpheli girdide: Besleme durdurulur; sürecin güvenli durumu budur. Alarm verilir ve operatör kontrol eder. YZ'nin kararı **kullanılmaz** [Y29] SEKTÖR-TİPİK.

Durum	Eylem	Neden
Normal kare, model OK (p = 0,05)	**GEÇ**	model
Normal kare, model RED (p = 0,55; düşük güven)	**AYIR**	model
Normal kare, model OK ama düşük güven (p = 0,45)	**OPERATÖRE SOR**	model
Kare 900 ms eski (yayın gecikti ya da tekrar oynatılıyor)	**BESLEMEYİ DURDUR**	kare eski
Aynı kare 5 kez üst üste (donmuş / sahte görüntü)	**BESLEMEYİ DURDUR**	donmuş kare
Lens kapalı (parlaklık 0,02)	**BESLEMEYİ DURDUR**	parlaklık sınır dışı, bulanık / kapalı lens
Model OK, metal dedektörü metal görüyor	**AYIR**	metal dedektörü YZ ile çelişiyor (bağımsız kanal kazanır)

Sınır: Bu basit kontroller, modeli kandırmak için görüntüye yerleştirilmiş bir desen gibi hedefli saldırıları yakalamaz. Bu yüzden asıl güvence ikinci kanal ve mimaridir. Kamera ile YZ arasındaki bağlantı, LAB 2'deki hat kumanda bölgesinde (Z4) tutulmalı ve dışarıdan erişilmemelidir ÇIKARIM.

3 Confidence düştü: Model "emin değilim" diyor. Eşik ne olmalı? Altında: dur / insana sor / fallback?

Eşik, bir hedeften türetilir; tahminle seçilmez. Kaçak hedefi (kusurların ≤ %1'i) ve operatör kapasitesi (parçaların ≤ %5'i) tanımlanır. Bu iki koşulu sağlayan en düşük eşik seçilir. Bu kümede eşik yalnız YZ ile **0,61**, metal dedektörüyle **0,56** çıkıyor.



Eşiğin altında:

- **OK → operatöre sor.** Besleme o parça için bekler; operatör onaylar ya da ayırır.
- **RED → ayır.** Güven düşük olsa da güvenli tarafa gidilir.
- **Operatör yoksa ya da cevap gelmezse → fallback:** Parça ayrılır ve besleme durur. Sistem hiçbir zaman "emin değilim ama geçsin" demez.

Eşik bir kez seçilip bırakılmaz. Sahadaki gerçek kaçak oranı izlenir (monitoring). Malzeme değişir de model kayarsa eşik yeniden hesaplanır.

Karar tablosu — AI Fail-Safe Decision

Başarısızlık	Tanımlanan güvenli davranış	Kavram
Model yanlış sınıflandırdı	OK yalnız güven eşiğinin üstünde otomatik geçer; YZ'den bağımsız metal dedektörü YZ'yi geçersiz kılar; RED her güvende ayırır; sahada kaçak oranı izlenir.	confidence threshold · fail-safe · monitoring
Girdi manipüle edildi	Kare yaşı, donmuş kare, parlaklık, keskinlik ve bağımsız sensörle tutarlılık kontrol edilir. Şüphede YZ kararı kullanılmaz, besleme durur ve alarm verilir.	monitoring · fail-safe · fallback
Confidence düştü	Eşiğin altında OK otomatik verilmez, operatöre sorulur. Operatör yoksa parça ayrılır ve besleme durur.	confidence threshold · human override · fallback

YZ'nin hız önerisi ve emniyet

YZ, sağlam ve küçük parçalarda dozaj hızını artırmayı önerebilir. LAB 4'ün bulgusu burada da geçerli: **güvenli mesafe bant hızına dayanıyor**. YZ'nin önerisi bu yüzden iki yerde sınırlanır:

1. **Normal kumanda:** Öneriyi nominal hızda keser.
2. **Emniyet tarafındaki hız izleme:** Normal kumanda atlatılsa bile bağımsız bir üst sınır uygular (LAB 4, G4).

YZ hız isteği	Kumanda keser + hız izleme	Kumanda atlatıldı + hız izleme	Kumanda atlatıldı, hız izleme yok
0,5 ×	0,219 m	0,219 m	0,219 m
1,0 ×	0,500 m	0,500 m	0,500 m
1,5 ×	0,500 m	0,630 m	0,844 m
2,0 ×	0,500 m	0,630 m	1,250 m
3,0 ×	0,500 m	0,630 m	2,225 m ✖

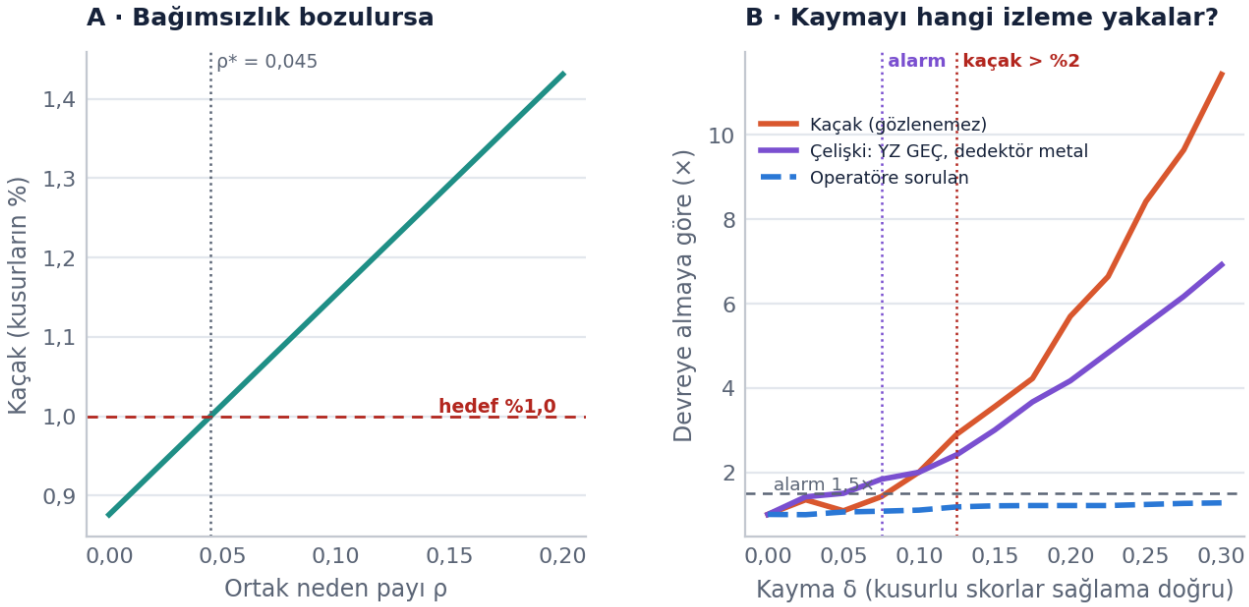
Acil stop mesafesi, SS1-t ile; güvenli mesafe 1,50 m. ✖ = el besleme ağzına ulaşır.

Hız izleme olmadan, YZ'nin 3 katlık bir isteği acil stop mesafesini 2,225 m'ye çıkarıyor ve el besleme ağzına ulaşıyor. Bu test `xfail(strict=True)` ile işaretli: gereksinimi karşılamayan bir düzende kalması beklenen sonuçtur.

YZ'nin sahadaki ömrü: kayma ve izleme

Devreye almadaki doğrulama, modelin sahada aynı kalacağını varsaymaz. Yeni bir yabancı cisim türü (ör. boyalı metal, plastik kaplı parça) modelin kusurlu parçalara verdiği skorları sağlam parçalarınkine doğru kaydırabilir (7.5) VARSAYIM (V26). Kaçak oranı sahada **gözlenemez**: kaçan parçayı kimse görmez. Gözlenebilen iki oran var: operatöre sorulan parçalar ve YZ'nin GEÇ dediği hâlde metal dedektörünün metal gördüğü parçalar (çelişki).

YZ + metal dedektörü: ortak neden ve kayma (sentetik veri)



Kayma δ	Kaçak	Operatöre sorulan	Çelişki (YZ GEÇ, dedektör metal)	Çelişki alarmı	Operatör yükü alarmı
0,000	%0,88	%0,78	5,7 / 10 000	—	—
0,050	%0,95	%0,83	8,5 / 10 000	—	—
0,100	%1,76 ✖	%0,86	11,4 / 10 000	**alarm**	—
0,150	%3,11 ✖	%0,95	17,1 / 10 000	**alarm**	—
0,200	%4,99 ✖	%0,95	23,8 / 10 000	**alarm**	—
0,250	%7,37 ✖	%0,97	31,3 / 10 000	**alarm**	—
0,300	%10,01 ✖	%1,00	39,4 / 10 000	**alarm**	—

Alarm: izlenen oran devreye almadaki değerinin 1,5 katına çıkınca. ✖ = kaçak hedefi (%1,0) aşıldı.

- Kaçak hedefi aşılıyor: δ = 0,025
- Kaçak hedefin 2 katını aşıyor: δ = 0,125
- Çelişki alarmı: δ = 0,075
- Operatör yükü alarmı: δ = taranan aralıkta yok
- **Operatör yükü kaymayı göstermiyor.** Kayma, modelin "emin olduğu" kararları yanlış yöne çekiyor; emin olmadığı parça sayısı pek değişmiyor (`test_operator_yuku_kaymayi_gostermez`).
- **Çelişki oranı gösteriyor.** Bağımsız kanal yalnızca kaçağı yakalamıyor, aynı zamanda YZ için bir **izleme sinyali** üretiyor. Alarm, kaçak hedefin iki katına çıkmadan çalıyor (`test_celiski_alarmi_kacak_toleransi_asilmadan_once_calar`).
- **Ama hedefin kendisi korunamıyor.** Devreye almada kaçak oranı hedefe çok yakın (%0,88'e karşı %1). Kayma başlar başlamaz hedef aşılabiliyor ve hiçbir izleme ondan önce davranamıyor. Bu test `xfail(strict=True)` : eşiği hedefe bu kadar yakın seçmek, sahada marj bırakmıyor **ÇIKARIM**.

Gereksinimler (föyün tarafsızlık ilkesiyle: çözüm değil gereksinim; her biri bir teste bağlı):

#	Gereksinim ve testi
A1	YZ'nin hiçbir çıktısı emniyet fonksiyonlarını (SF1–SF4) etkilememeli; acil stop her YZ çıktısında güvenli duruma götürmelidir. Test: `test_yz_hicbir_ciktiyla_acil_stopu_etkilemez`
A2	YZ makineyi güvenli durumdan çıkaramamalı ve çalıştıramamalı; YZ yalnızca durdurma isteyebilir. Test: `test_yz_guvenli_durumdan_makineyi_calistiramaz`
A3	YZ'nin hız önerisi normal kumandada sınırlanmalı; emniyet tarafı bu sınırdan bağımsız bir hız üst sınırı uygulamalıdır. Test: `test_yz_hiz_istegi_normal_kumandada_kesilir` , `test_kumanda_atlatilsa_da_hiz_izleme_eli_korur`
A4	Güven eşiği, tanımlı bir kaçak hedefinden ve operatör kapasitesinden türetilmeli; eşiğin altında OK otomatik verilmemelidir. Test: `test_secilen_esik_kacak_hedefini_operator_kapasitesi_icinde_karsilar` , `test_esigin_altinda_ok_otomatik_verilmez`
A5	Şüpheli girdide YZ kararı kullanılmamalı ve süreç güvenli duruma (besleme durur) geçmelidir. Test: `test_supheli_girdide_besleme_durur`
A6	YZ'den bağımsız bir kanal YZ ile çelişirse güvenli taraf (ayır) seçilmelidir. Test: `test_metal_dedektoru_yz_ile_celisirse_parca_ayrilir`
A7	YZ'nin sahadaki performansı, kaçak oranına dayanmayan gözlenebilir bir sinyalle (ör. bağımsız kanalla çelişki oranı) izlenmeli; kayma, kaçak tanımlı bir toleransı aşmadan alarm üretmelidir. İkinci kanalın ortak neden payı devreye almada ölçülmelidir. Test: `test_celiski_alarmi_kacak_toleransi_asilmadan_once_calar` , `test_ortak_neden_esigi_asilinca_kacak_hedefi_asilir`

> **Tarafsızlık notu:** Metal dedektörü, güven eşiği değerleri ve girdi kontrolleri yalnızca gereksinimlerin test edilebildiğini göstermek içindir; bir ürün ya da tasarım önerisi değildir. Tasarım kararı firmaya aittir.

Kanıt: simülasyon (LAB 7)

Eğitim amaçlı modeldir. Skorlar Beta dağılımlarından üretilmiş sentetik verilerdir; gerçek bir YZ modelinin performansı değildir. ISO/IEC TR 5469 değerlendirmesinin yerine geçmez.

Ne gösterildi?

- Güven eşiği tarandı; kaçak oranı azalırken operatör yükünün arttığı ve hedefin hangi eşikte karşılandığı gösterildi.
- Bağımsız ikinci kanalın hem kaçak oranını hem de operatör yükünü azalttığı gösterildi.
- Girdi kontrolleri, şüpheli her girdide beslemeyi durdurdu.
- YZ'nin hiçbir çıktısı emniyet durum makinesini (LAB 1) etkilemedi; hız önerisi, emniyet tarafındaki hız izlemeyle sınırlandı (LAB 4).
- İkinci kanalın katkısı bağımsızlığa bağlı: küçük bir ortak neden payı hedefi bozuyor (7.4).

• Sahadaki kaymayı operatör yükü değil, YZ ile bağımsız kanal arasındaki çelişki oranı yakalıyor (7.5).

Test sonuçları:

#	Test (dosya)	Sonuç
1	`test_esiksiz_model_kacak_hedefini_karsilamaz` (test_lab7_ai)	✔ geçti
2	`test_secilen_esik_kacak_hedefini_operator_kapasitesi_icinde_karsilar` (test_lab7_ai)	✔ geçti
3	`test_bagimsiz_metal_dedektoru_operator_yukunu_azaltir` (test_lab7_ai)	✔ geçti
4	`test_esik_arttikca_kacak_azalir_operator_yuku_artar` (test_lab7_ai)	✔ geçti
5	`test_red_guven_dusuk_olsa_da_ayirir` (test_lab7_ai)	✔ geçti
6	`test_normal_girdi_supheli_degil` (test_lab7_ai)	✔ geçti
7	`test_supheli_girdide_besleme_durur` (test_lab7_ai)	✔ geçti (4/4)
8	`test_metal_dedektoru_yz_ile_celisirse_parca_ayrilir` (test_lab7_ai)	✔ geçti
9	`test_esigin_altinda_ok_otomatik_verilmez` (test_lab7_ai)	✔ geçti (3/3)
10	`test_yz_hicbir_ciktiyla_acil_stopu_etkilemez` (test_lab7_ai)	✔ geçti (12/12)
11	`test_yz_guvenli_durumdan_makineyi_calistiramaz` (test_lab7_ai)	✔ geçti (4/4)
12	`test_yz_hiz_istegi_normal_kumandada_kesilir` (test_lab7_ai)	✔ geçti (5/5)
13	`test_kumanda_atlatilsa_da_hiz_izleme_eli_korur` (test_lab7_ai)	✔ geçti (5/5)
14	`test_hiz_izleme_olmadan_da_eli_korur` (test_lab7_ai)	✖ kaldı (beklenen)
15	`test_bagimsiz_hatalarda_ikinci_kanal_kacak_hedefini_saglar` (test_lab7_ai)	✔ geçti
16	`test_ortak_neden_payi_artinca_kacak_artar` (test_lab7_ai)	✔ geçti
17	`test_ortak_neden_esigi_asilinca_kacak_hedefi_asilir` (test_lab7_ai)	✔ geçti
18	`test_kaymasiz_veri_devreye_almadaki_kumeyle_aynidir` (test_lab7_ai)	✔ geçti
19	`test_kusurlu_skorlar_saglama_kaydikca_kacak_hedefi_asilir` (test_lab7_ai)	✔ geçti
20	`test_operator_yuku_kaymayi_gostermez` (test_lab7_ai)	✔ geçti
21	`test_celiski_alarmi_kacak_toleransi_asilmadan_once_calar` (test_lab7_ai)	✔ geçti
22	`test_celiski_alarmi_kacak_hedefi_asilmadan_once_calar` (test_lab7_ai)	✖ kaldı (beklenen)

Yorum: "Prediction ≠ Safety" ilkesinin sayısal karşılığı şu: model ne kadar iyi olursa olsun %0 kaçak vermiyor. Güvenlik, modelin etrafındaki kurallardan (eşik, ikinci kanal, girdi kontrolleri) ve YZ'nin hiç dokunamadığı emniyet zincirinden geliyor.

Ek sorular: zafiyetler ve nasıl kapatılır

Föy dışıdır; kullanıcı isteğiyle eklendi. Öneriler genel sektör pratiğidir; Efo'nun bunları uygulayıp uygulamadığı bilinmiyor.

E1 Kamerayı besleme ağzına yaklaşan insanı algılamak için de kullanabilir miyiz?

- **Risk:** YZ ile insan algılama, ışık perdesi ya da çekme ipinin yerine geçerse emniyet fonksiyonu bir tahmine bağlanır. Model bir insanı kaçırırsa sonuç ölümcül olabilir (LAB 1, S2) ÇIKARIM.
- **Kapatma:** YZ ile insan algılama yalnızca **ek bir uyarı** olarak kullanılabilir (ör. sesli ikaz, bandı yavaşlatma isteği). SF1–SF4 kablolu zincirde kalır. Emniyet fonksiyonu YZ'ye verilecekse 2023/1230 Ek I Kısım A kapsamında üçüncü taraf uygunluk değerlendirmesi gerekir [Y27, Y29] SEKTÖR-TİPİK.

E2 Model güncellemesi bir yazılım değişikliği midir?

- **Zafiyet:** Yeni bir model dosyası, firmware yaması kadar davranışı değiştirir. Ama çoğu zaman "sadece veri" sanılıp test edilmeden yüklenir ÇIKARIM.
- **Kapatma:** Model sürümünü SBOM'a eklemek ve LAB 5'teki akıştan geçirmek: önce test kümesinde kaçak oranı ölçülür, eşik yeniden hesaplanır, sonra bakım penceresinde yüklenir. Model dosyasının özet değeri (hash) kaydedilir ve olay anında referansla karşılaştırılır (LAB 6) SEKTÖR-TİPİK.

E3 YZ'nin kararları kaydedilmeli mi?

- **Neden önemli:** Bir bıçak hasarından sonra "YZ o parçaya ne dedi, güveni neydi?" sorusu cevaplanabilmeli. 2023/1230 Ek III 1.2.1, kendi kendine gelişen davranışlı emniyet sistemleri için emniyetle ilgili karar sürecinin kaydını istiyor [Y27] SEKTÖR-TİPİK. Bu proje emniyet fonksiyonu olmadığı için kapsamda değil, ama aynı pratik izlemeyi mümkün kılar ÇIKARIM.
- **Kapatma:** Her kararın zamanını, model sürümünü, skorunu ve eylemini kaydetmek. Kaçak ve operatör onay oranını vardiya bazında izlemek; kayma görülürse eşiği yeniden hesaplamak.

Ek — Kod

efo_sim/lab7_ai.py

```python
"""LAB 7 — YZ fail-safe kararı (eğitim amaçlı; proje aşaması değerlendirmesi).

Senaryo: besleme bandına YZ'li kamera ile yabancı cisim / aşırı boy parça kontrolü eklenmesi
planlanıyor. Kamera → YZ → OK / RED → ayırma klapesi → dozaj hızı.

7.1 Yanlış sınıflandırma: sentetik skorlarla güven eşiği taraması; kaçak oranı ve operatöre
    sorulan oran. YZ'den bağımsız metal dedektörü ikinci kanal olarak eklenir.
7.2 Girdi manipülasyonu: kare yaşı, donmuş kare, parlaklık, keskinlik ve metal dedektörüyle
    tutarsızlık; şüpheli girdide besleme durur.
7.3 Güven düşük: eşiğin altında OK otomatik verilmez, operatöre sorulur.
7.4 Ortak neden: YZ ile metal dedektörünün hataları tam bağımsız değilse kaçak oranı ne olur?
7.5 Kayma: kusurlu parçaların skorları sağlamlara doğru kayarsa kaçak artar; bunu hangi izleme
    yakalar? Operatör yükü değil, YZ ile metal dedektörü arasındaki çelişki oranı.
YZ emniyet zincirinde değildir: hiçbir çıktısı acil stopu etkilemez; hız isteği normal kumandada
kesilir ve emniyet tarafındaki hız izleme (LAB 4) bağımsız bir üst sınır uygular.

Çalıştırma (repo kökünden):  python -m efo_sim.lab7_ai
"""
import math
import random
from functools import lru_cache

from efo_sim import stil
from efo_sim import lab1_safety as lab1
from efo_sim.params import OUT_DIR, bolum_metni, d, sayi

GEC, AYIR, SOR, DURDUR = "GEÇ", "AYIR", "OPERATÖRE SOR", "BESLEMEYİ DURDUR"


# ---------------------------------------------------------------- 7.1 Yanlış sınıflandırma
@lru_cache(maxsize=None)
def veri_uret(kayma=0.0):
    """(kusurlu, metal, p_kusur) üçlüleri; dağılımlar params.yaml'da. Önbellekli: params değişirse cache_clear().

    kayma (δ): kusurlu skor dağılımının parametreleri sağlamınkine doğru δ oranında kayar (7.5).
    Aynı tohum kullanılır; δ = 0 devreye almadaki kümedir."""
    r = random.Random(d("lab7.tohum"))
    (ak, bk), (as_, bs) = d("lab7.skor_kusurlu"), d("lab7.skor_saglam")
    veri = []
    for _ in range(d("lab7.ornek_sayisi")):
        kusurlu = r.random() < d("lab7.kusur_orani")
        metal = kusurlu and r.random() < d("lab7.metal_payi")
        a, b = (ak + kayma * (as_ - ak), bk + kayma * (bs - bk)) if kusurlu else (as_, bs)
        veri.append((kusurlu, metal, r.betavariate(a, b)))
    return tuple(veri)


def guven(p):
    return max(p, 1 - p)

```

```python

def model_karari(p, esik):
    """RED her güvende ayırır (güvenli taraf); OK yalnız güven eşiği aşılınca otomatik geçer."""
    if p >= d("lab7.karar_siniri"):
        return AYIR
    return GEC if guven(p) >= esik else SOR


def md_kacirma(ortak_neden=0.0):
    """YZ metal parçayı kaçırdığında dedektörün de kaçırma olasılığı.
    ρ payında hatalar ortak nedenlidir (ikisi birlikte kaçırır); kalanında bağımsızdır (7.4)."""
    q = d("lab7.md_kacirma_orani")
    return ortak_neden + (1 - ortak_neden) * q


def metrikler(veri, esik, md=False, ortak_neden=0.0):
    """Kaçak = kusurlu parçanın otomatik GEÇ alması. md=True: metal dedektörü metal kaçakları yakalar.
    celiski: YZ GEÇ dediği hâlde dedektörün metal gördüğü parçaların oranı (sahada gözlenebilir)."""
    kusurlu = [x for x in veri if x[0]]
    kacak = [x for x in kusurlu if model_karari(x[2], esik) == GEC]
    kacak_oran = len(kacak) / len(kusurlu)
    q = md_kacirma(ortak_neden)
    if md:
        # metal kaçağı ancak dedektör de kaçırırsa geçer (beklenen değer)
        kacak_oran = sum(q if x[1] else 1 for x in kacak) / len(kusurlu)
    sor = sum(model_karari(x[2], esik) == SOR for x in veri) / len(veri)
    yanlis_ret = sum(not x[0] and model_karari(x[2], esik) == AYIR for x in veri) / len(veri)
    celiski = sum(1 - q for x in kacak if x[1]) / len(veri)
    return {"kacak": kacak_oran, "sor": sor, "yanlis_ret": yanlis_ret, "celiski": celiski}


@lru_cache(maxsize=1)
def esik_taramasi():
    veri = veri_uret()
    alt, ust = d("lab7.esik_araligi")
    n = d("lab7.esik_adim_sayisi")
    return tuple((e, metrikler(veri, e), metrikler(veri, e, md=True))
                 for e in (alt + (ust - alt) * i / (n - 1) for i in range(n)))


def esik_sec(md=False):
    """Kaçak hedefini karşılayan en düşük eşik (operatör yükü en az). Yoksa None."""
    for e, yalniz, birlikte in esik_taramasi():
        if (birlikte if md else yalniz)["kacak"] <= d("lab7.hedef_kacak_orani"):
            return e
    return None


# ---------------------------------------------------------------- 7.2 Girdi manipülasyonu
def girdi_supheli(kare, onceki_hashler=()):
    """Şüphe nedenleri listesi; boşsa girdi kabul edilir."""
    g = d("lab7.girdi_kontrolleri")
    nedenler = []
    if kare["yas_ms"] > g["azami_kare_yasi_ms"]:
        nedenler.append("kare eski")
    son = list(onceki_hashler)[-(g["donuk_kare_sayisi"] - 1):]
    if len(son) == g["donuk_kare_sayisi"] - 1 and all(h == kare["hash"] for h in son):
        nedenler.append("donmuş kare")
    alt, ust = g["parlaklik_araligi"]
    if not alt <= kare["parlaklik"] <= ust:
        nedenler.append("parlaklık sınır dışı")
    if kare["keskinlik"] < g["asgari_keskinlik"]:
        nedenler.append("bulanık / kapalı lens")
    return nedenler


def guvenli_davranis(p, kare, onceki_hashler=(), metal_dedektoru=False, esik=None):
    """YZ çıktısının etrafındaki kural katmanı. Döner: (eylem, neden)."""
    nedenler = girdi_supheli(kare, onceki_hashler)
    if nedenler:
        return DURDUR, ", ".join(nedenler)
    karar = model_karari(p, esik if esik is not None else esik_sec())
    if metal_dedektoru and karar != AYIR:
        return AYIR, "metal dedektörü YZ ile çelişiyor (bağımsız kanal kazanır)"
    return karar, "model"


# ---------------------------------------------------------------- 7.3 Hız ve emniyet bağımsızlığı
```

```python
def normal_kumanda_hizi(istek):
    """Normal kumanda YZ'nin dozaj hızı isteğini üst sınırda keser (× nominal)."""
    return min(istek, d("lab7.hiz_istegi_ust_siniri"))


def acil_stop_mesafesi(istek, kumanda_kesiyor=True, hiz_izleme=True):
    """YZ hız istediğinde acil stop mesafesi. kumanda_kesiyor=False: normal kumanda atlatıldı."""
    carpan = normal_kumanda_hizi(istek) if kumanda_kesiyor else istek
    if hiz_izleme:
        carpan = min(carpan, d("lab4.hiz_izleme_esigi_carpani"))
    return lab1.durma_mesafesi(d("lab1.bant_hizi") * carpan, 1, ss1_emniyette=True)


def hat_cevrimi(durum, g, yz_eylemi):
    """Bir kumanda çevrimi: emniyet durum makinesi önce, YZ yalnız normal stop isteyebilir."""
    durum = lab1.sonraki_durum(durum, g)
    if durum == lab1.CALISIYOR and yz_eylemi == DURDUR:
        return lab1.DURUYOR
    return durum


# ---------------------------------------------------------------- 7.4 Ortak neden
def _tarama(yol):
    alt, ust, n = d(yol)
    return [alt + (ust - alt) * i / (n - 1) for i in range(n)]


def ortak_neden_esigi():
    """Seçilen eşikte (YZ + dedektör) kaçak hedefini aşan en küçük ortak neden payı ρ*.
    Kaçak ρ'ya göre doğrusaldır: kacak(ρ) = (yalnız_YZ_kaçak_metal_dışı + metal_kaçak · q(ρ)) / kusurlu."""
    veri, e = veri_uret(), esik_sec(md=True)
    kusurlu = [x for x in veri if x[0]]
    kacak = [x for x in kusurlu if model_karari(x[2], e) == GEC]
    metal_disi, metal = sum(not x[1] for x in kacak), sum(x[1] for x in kacak)
    q, hedef = d("lab7.md_kacirma_orani"), d("lab7.hedef_kacak_orani")
    if not metal:
        return None
    rho = (hedef * len(kusurlu) - metal_disi - metal * q) / (metal * (1 - q))
    return max(rho, 0.0)


def ortak_neden_taramasi():
    veri, e = veri_uret(), esik_sec(md=True)
    return [(rho, metrikler(veri, e, md=True, ortak_neden=rho)) for rho in _tarama("lab7.ortak_neden_taramasi")]


# ---------------------------------------------------------------- 7.5 Kayma ve izleme
def kayma_taramasi():
    """Her δ için seçilen eşikte (YZ + dedektör) metrikler. Eşik devreye almada seçildiği gibi kalır."""
    e = esik_sec(md=True)
    return [(k, metrikler(veri_uret(k), e, md=True)) for k in _tarama("lab7.kayma_taramasi")]


def ilk_kayma(kosul):
    return next((k for k, r in kayma_taramasi() if kosul(r)), None)


def izleme_sonucu():
    """Hangi δ'da kaçak hedef ve tolerans aşılıyor, hangi izleme hangi δ'da alarm veriyor?"""
    taban = kayma_taramasi()[0][1]
    c, hedef = d("lab7.izleme_alarm_carpani"), d("lab7.hedef_kacak_orani")
    return {
        "hedef_asim": ilk_kayma(lambda r: r["kacak"] > hedef),
        "tolerans_asim": ilk_kayma(lambda r: r["kacak"] > hedef * d("lab7.kayma_tolerans_carpani")),
        "alarm_celiski": ilk_kayma(lambda r: r["celiski"] >= c * taban["celiski"]),
        "alarm_sor": ilk_kayma(lambda r: r["sor"] >= c * taban["sor"]),
        "taban": taban,
    }


# ---------------------------------------------------------------- Çıktılar
def esik_md():
    veri = veri_uret()
    e1, e2 = esik_sec(), esik_sec(md=True)
    hedef = d("lab7.hedef_kacak_orani")
    satirlar = [("Eşiksiz (her OK geçer)", d("lab7.esik_araligi")[0], False),
                ("Güven eşiği", e1, False),
                ("Güven eşiği + metal dedektörü", e2, True)]
```

```
    s = ["| Düzen | Güven eşiği | Kaçak (kusurların) | Operatöre sorulan (tüm parçaların) | Yanlış ret | Hedef ≤
"
         + f"%{sayi(hedef * 100, 1)} |", "|---|---|---|---|---|---|"]
    for ad, e, md in satirlar:
        if e is None:
            s.append(f"| {ad} | bulunamadı | — | — | — | ✖ |")
            continue
        r = metrikler(veri, e, md=md)
        s.append(f"| {ad} | {sayi(e)} | %{sayi(r['kacak'] * 100, 2)} | %{sayi(r['sor'] * 100, 2)} | "
                 f"%{sayi(r['yanlis_ret'] * 100, 2)} | {'✔' if r['kacak'] <= hedef else '✖'} |")
    kap = d("lab7.insan_kapasitesi")
    s += ["", f"Sentetik küme: {d('lab7.ornek_sayisi')} parça, kusur oranı %{sayi(d('lab7.kusur_orani') * 100,
1)}, "
              f"kusurların %{sayi(d('lab7.metal_payi') * 100, 0)}'ı metal. Operatör kapasitesi tüm parçaların "
              f"%{sayi(kap * 100, 0)}'i."]
    return "\n".join(s) + "\n"


def girdi_md():
    normal = {"yas_ms": 40, "hash": "a1", "parlaklik": 0.5, "keskinlik": 0.6}
    n = d("lab7.girdi_kontrolleri")["donuk_kare_sayisi"]
    durumlar = [
        ("Normal kare, model OK (p = 0,05)", 0.05, normal, (), False),
        ("Normal kare, model RED (p = 0,55; düşük güven)", 0.55, normal, (), False),
        ("Normal kare, model OK ama düşük güven (p = 0,45)", 0.45, normal, (), False),
        ("Kare 900 ms eski (yayın gecikti ya da tekrar oynatılıyor)", 0.05, normal | {"yas_ms": 900}, (), False),
        (f"Aynı kare {n} kez üst üste (donmuş / sahte görüntü)", 0.05, normal, ["a1"] * (n - 1), False),
        ("Lens kapalı (parlaklık 0,02)", 0.05, normal | {"parlaklik": 0.02, "keskinlik": 0.05}, (), False),
        ("Model OK, metal dedektörü metal görüyor", 0.05, normal, (), True),
    ]
    esik = esik_sec()
    s = ["| Durum | Eylem | Neden |", "|---|---|---|"]
    for ad, p, kare, gecmis, md in durumlar:
        eylem, neden = guvenli_davranis(p, kare, gecmis, md, esik=esik)
        s.append(f"| {ad} | **{eylem}** | {neden} |")
    return "\n".join(s) + "\n"


def hiz_md():
    s = ["| YZ hız isteği | Kumanda keser + hız izleme | Kumanda atlatıldı + hız izleme | Kumanda atlatıldı, hız
izleme yok |",
         "|---|---|---|---|"]
    sinir = d("lab1.guvenli_mesafe")

    def hucre(x):
        return f"{sayi(x, 3)} m" + (" **✖**" if lab1.el_besleme_agzina_ulasir_mi(x) else "")

    for istek in d("lab7.hiz_istegi_taramasi"):
        s.append(f"| {sayi(istek, 1)} × | {hucre(acil_stop_mesafesi(istek))} | "
                 f"{hucre(acil_stop_mesafesi(istek, kumanda_kesiyor=False))} | "
                 f"{hucre(acil_stop_mesafesi(istek, kumanda_kesiyor=False, hiz_izleme=False))} |")
    s += ["", f"Acil stop mesafesi, SS1-t ile; güvenli mesafe {sayi(sinir)} m. ✖ = el besleme ağzına ulaşır."]
    return "\n".join(s) + "\n"


def _grafik_esik(yol):
    """Logaritmik y ekseni: %1'in altındaki kaçak oranı ile %100'e yaklaşan operatör yükü aynı grafikte
okunur."""
    plt = stil.mpl()
    import matplotlib.ticker

    R = stil.RENK
    tarama = esik_taramasi()
    xs = [t[0] for t in tarama]
    seriler = [
        ([t[1]["kacak"] * 100 for t in tarama], R["giris"], "-", "Kaçak: yalnız YZ"),
        ([t[2]["kacak"] * 100 for t in tarama], R["iyi"], "-", "Kaçak: YZ + metal dedektörü"),
        ([t[1]["sor"] * 100 for t in tarama], R["bilgi"], "--", "Operatöre sorulan"),
    ]
    fig, ax = plt.subplots(figsize=(7.2, 3.7))
    stil.eksen(ax, izgara="both")
    ax.set_yscale("log", nonpositive="mask")
    for ys, renk, ls, ad in seriler:
        ax.plot(xs, [y if y > 0 else float("nan") for y in ys], color=renk, lw=2, ls=ls, label=ad)
    pozitif = [y for ys, *_ in seriler for y in ys if y > 0]
    ax.set_ylim(min(pozitif) / 2, max(pozitif) * 2)
    hedef, kap = d("lab7.hedef_kacak_orani") * 100, d("lab7.insan_kapasitesi") * 100
    for y, renk, metin in ((hedef, R["kritik"], f"kaçak hedefi %{sayi(hedef, 1)}"),
```

```
                           (kap, R["bilgi"], f"operatör kapasitesi %{sayi(kap, 0)}")):
        ax.axhline(y, color=renk, lw=1, ls=(0, (4, 3)))
        ax.text(xs[-1], y, metin + " ", color=renk, fontsize=7.2, fontweight="bold", ha="right", va="bottom")
    secimler = sorted(((esik_sec(), R["giris"], "yalnız YZ"), (esik_sec(md=True), R["iyi"], "YZ + dedektör")),
                      key=lambda t: t[0] if t[0] is not None else 0)
    for sira, (e, renk, ad) in enumerate(secimler):
        if e is not None:  # soldaki eşiğin etiketi sola, sağdakinin sağa: çakışmasınlar
            ax.axvline(e, color=renk, lw=1, alpha=0.7)
            ax.text(e, ax.get_ylim()[1], f" eşik {sayi(e)} \n ({ad}) ", color=renk, fontsize=6.8,
                    va="top", ha="right" if sira == 0 else "left", fontweight="bold")
    ax.xaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(lambda x, _: sayi(x)))
    ax.yaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(
        lambda y, _: "%" + sayi(y, max(0, -math.floor(math.log10(y))))))
    ax.tick_params(which="both", length=0)
    ax.yaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
    ax.set_xlabel("Güven eşiği (OK'nin otomatik geçmesi için)")
    ax.set_ylabel("Oran (logaritmik)")
    ax.set_title("Güven eşiği: kaçak oranı ve operatör yükü (sentetik veri)")
    ax.legend(loc="lower right")
    fig.savefig(yol)
    plt.close(fig)


def _yuzde(x, basamak=2):
    return f"%{sayi(x * 100, basamak)}"


def ortak_neden_md():
    hedef, rho_y = d("lab7.hedef_kacak_orani"), ortak_neden_esigi()
    s = ["| Ortak neden payı ρ | Dedektörün kaçırma olasılığı | Kaçak (kusurların) | Hedef ≤ " + _yuzde(hedef,
1) + " |",
         "|---|---|---|---|"]
    for rho, r in ortak_neden_taramasi()[::4]:
        s.append(f"| {sayi(rho)} | {_yuzde(md_kacirma(rho), 1)} | {_yuzde(r['kacak'])} | "
                 f"{'✔' if r['kacak'] <= hedef else '**✖**'} |")
    s += ["", f"Seçilen eşik {sayi(esik_sec(md=True))} (YZ + metal dedektörü). Kaçak hedefi, ortak neden payı "
              f"**ρ* = {sayi(rho_y, 3)}** değerini aştığında aşılıyor: YZ'nin kaçırdığı her {round(1 / rho_y)}
metal "
              "parçadan birini dedektörün de aynı nedenle kaçırması yetiyor."]
    return "\n".join(s) + "\n"


def kayma_md():
    iz, hedef = izleme_sonucu(), d("lab7.hedef_kacak_orani")
    taban, c = iz["taban"], d("lab7.izleme_alarm_carpani")
    s = ["| Kayma δ | Kaçak | Operatöre sorulan | Çelişki (YZ GEÇ, dedektör metal) | Çelişki alarmı | Operatör
yükü alarmı |",
         "|---|---|---|---|---|---|"]
    for k, r in kayma_taramasi()[::2]:
        s.append(f"| {sayi(k, 3)} | {_yuzde(r['kacak'])}{' **✖**' if r['kacak'] > hedef else ''} | "
                 f"{_yuzde(r['sor'])} | {sayi(r['celiski'] * 10000, 1)} / 10 000 | "
                 f"{'**alarm**' if r['celiski'] >= c * taban['celiski'] else '–'} | "
                 f"{'**alarm**' if r['sor'] >= c * taban['sor'] else '–'} |")

    def km(x):
        return sayi(x, 3) if x is not None else "taranan aralıkta yok"

    s += ["", f"Alarm: izlenen oran devreye almadaki değerinin {sayi(c, 1)} katına çıkınca. ✖ = kaçak hedefi "
              f"({_yuzde(hedef, 1)}) aşıldı.", "",
          f"- Kaçak hedefi aşılıyor: δ = {km(iz['hedef_asim'])}",
          f"- Kaçak hedefin {sayi(d('lab7.kayma_tolerans_carpani'), 0)} katını aşıyor: δ =
{km(iz['tolerans_asim'])}",
          f"- Çelişki alarmı: δ = {km(iz['alarm_celiski'])}",
          f"- Operatör yükü alarmı: δ = {km(iz['alarm_sor'])}"]
    return "\n".join(s) + "\n"


def _grafik_ortak_neden_kayma(yol):
    plt = stil.mpl()
    import matplotlib.ticker

    R = stil.RENK
    hedef = d("lab7.hedef_kacak_orani")
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(7.6, 3.2), gridspec_kw={"wspace": 0.32})

    # A: ortak neden
    on = ortak_neden_taramasi()
    rho_y = ortak_neden_esigi()
```

```
    stil.eksen(a1)
    a1.plot([x for x, _ in on], [r["kacak"] * 100 for _, r in on], color=R["iyi"], lw=2)
    a1.axhline(hedef * 100, color=R["kritik"], lw=1, ls=(0, (4, 3)))
    a1.text(on[-1][0], hedef * 100, f"hedef %{sayi(hedef * 100, 1)} ", color=R["kritik"], fontsize=6.8,
            ha="right", va="bottom", fontweight="bold")
    a1.axvline(rho_y, color=R["ikincil"], lw=0.9, ls=":")
    a1.text(rho_y, a1.get_ylim()[1], f" ρ* = {sayi(rho_y, 3)}", color=R["ikincil"], fontsize=6.8, va="top")
    a1.set_xlabel("Ortak neden payı ρ")
    a1.set_ylabel("Kaçak (kusurların %)")
    a1.set_title("A · Bağımsızlık bozulursa", fontsize=9)
    a1.xaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(lambda x, _: sayi(x)))
    a1.yaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(lambda y, _: sayi(y, 1)))

    # B: kayma ve izleme (devreye almaya göre kat)
    kt, iz = kayma_taramasi(), izleme_sonucu()
    taban = iz["taban"]
    xs = [k for k, _ in kt]
    stil.eksen(a2)
    for anahtar, renk, ls, ad in (("kacak", R["giris"], "-", "Kaçak (gözlenemez)"),
                                  ("celiski", R["vurgu"], "-", "Çelişki: YZ GEÇ, dedektör metal"),
                                  ("sor", R["bilgi"], "--", "Operatöre sorulan")):
        a2.plot(xs, [r[anahtar] / taban[anahtar] for _, r in kt], color=renk, lw=2, ls=ls, label=ad)
    c = d("lab7.izleme_alarm_carpani")
    a2.axhline(c, color=R["ikincil"], lw=0.9, ls=(0, (4, 3)))
    a2.text(xs[0], c, f" alarm {sayi(c, 1)}×", color=R["ikincil"], fontsize=6.6, va="bottom")
    for k, renk, metin in ((iz["alarm_celiski"], R["vurgu"], "alarm"),
                           (iz["tolerans_asim"], R["kritik"], f"kaçak > %{sayi(hedef * d('lab7.kayma_tolerans_ca
rpani') * 100, 0)}")):
        if k is not None:
            a2.axvline(k, color=renk, lw=0.9, ls=":")
            a2.text(k, a2.get_ylim()[1], f" {metin}", color=renk, fontsize=6.6, va="top", fontweight="bold")
    a2.set_xlabel("Kayma δ (kusurlu skorlar sağlama doğru)")
    a2.set_ylabel("Devreye almaya göre (×)")
    a2.set_title("B · Kaymayı hangi izleme yakalar?", fontsize=9)
    a2.legend(loc="upper left", bbox_to_anchor=(0, 0.9), fontsize=6.4)
    a2.xaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(lambda x, _: sayi(x)))
    a2.yaxis.set_major_formatter(matplotlib.ticker.FuncFormatter(lambda y, _: sayi(y, 0)))
    fig.suptitle("YZ + metal dedektörü: ortak neden ve kayma (sentetik veri)", x=0.01, ha="left",
                 fontsize=10, fontweight="bold", y=1.03)
    fig.savefig(yol)
    plt.close(fig)


def main():
    OUT_DIR.mkdir(exist_ok=True)
    (OUT_DIR / "lab7_esik.md").write_text(esik_md(), encoding="utf-8")
    (OUT_DIR / "lab7_girdi.md").write_text(girdi_md(), encoding="utf-8")
    (OUT_DIR / "lab7_hiz.md").write_text(hiz_md(), encoding="utf-8")
    (OUT_DIR / "lab7_params.yaml").write_text(bolum_metni("lab7"), encoding="utf-8")
    _grafik_esik(OUT_DIR / "lab7_esik.png")
    (OUT_DIR / "lab7_ortak_neden.md").write_text(ortak_neden_md(), encoding="utf-8")
    (OUT_DIR / "lab7_kayma.md").write_text(kayma_md(), encoding="utf-8")
    _grafik_ortak_neden_kayma(OUT_DIR / "lab7_ortak_neden_kayma.png")
    print(esik_md())
    print(girdi_md())
    print(hiz_md())


if __name__ == "__main__":
    main()
```

efo_sim/tests/test_lab7_ai.py

```
"""LAB 7 gereksinimleri = testler (eğitim amaçlı model; sentetik veri)."""
import pytest

from efo_sim import lab1_safety as lab1
from efo_sim import lab7_ai as m

NORMAL = {"yas_ms": 40, "hash": "a1", "parlaklik": 0.5, "keskinlik": 0.6}
TUM_EYLEMLER = [m.GEC, m.AYIR, m.SOR, m.DURDUR]


# ---------------------------------------------------------------- 7.1 Yanlış sınıflandırma
def test_esiksiz_model_kacak_hedefini_karsilamaz():
    ilk = m.esik_taramasi()[0][1]
    assert ilk["kacak"] > m.d("lab7.hedef_kacak_orani")

```

```python

def test_secilen_esik_kacak_hedefini_operator_kapasitesi_icinde_karsilar():
    e = m.esik_sec()
    assert e is not None
    r = m.metrikler(m.veri_uret(), e)
    assert r["kacak"] <= m.d("lab7.hedef_kacak_orani")
    assert r["sor"] <= m.d("lab7.insan_kapasitesi")


def test_bagimsiz_metal_dedektoru_operator_yukunu_azaltir():
    assert m.esik_sec(md=True) <= m.esik_sec()
    veri = m.veri_uret()
    assert m.metrikler(veri, m.esik_sec(md=True))["sor"] <= m.metrikler(veri, m.esik_sec())["sor"]


def test_esik_arttikca_kacak_azalir_operator_yuku_artar():
    tarama = m.esik_taramasi()
    kacak = [t[1]["kacak"] for t in tarama]
    sor = [t[1]["sor"] for t in tarama]
    assert kacak == sorted(kacak, reverse=True)
    assert sor == sorted(sor)


def test_red_guven_dusuk_olsa_da_ayirir():
    assert m.model_karari(m.d("lab7.karar_siniri"), esik=0.99) == m.AYIR


# ---------------------------------------------------------------- 7.2 Girdi manipülasyonu
def test_normal_girdi_supheli_degil():
    assert m.girdi_supheli(NORMAL, ["x", "y", "z", "w"]) == []


@pytest.mark.parametrize("kare,gecmis,neden", [
    (NORMAL | {"yas_ms": 900}, (), "kare eski"),
    (NORMAL, ["a1"] * 4, "donmuş kare"),
    (NORMAL | {"parlaklik": 0.99}, (), "parlaklık sınır dışı"),
    (NORMAL | {"keskinlik": 0.01}, (), "bulanık / kapalı lens"),
])
def test_supheli_girdide_besleme_durur(kare, gecmis, neden):
    eylem, aciklama = m.guvenli_davranis(0.01, kare, gecmis, esik=0.6)
    assert eylem == m.DURDUR and neden in aciklama


def test_metal_dedektoru_yz_ile_celisirse_parca_ayrilir():
    assert m.guvenli_davranis(0.01, NORMAL, metal_dedektoru=True, esik=0.6)[0] == m.AYIR


# ---------------------------------------------------------------- 7.3 Güven düşük
@pytest.mark.parametrize("p", [0.40, 0.45, 0.49])
def test_esigin_altinda_ok_otomatik_verilmez(p):
    e = m.esik_sec()
    assert m.guven(p) < e
    assert m.model_karari(p, e) == m.SOR


# ---------------------------------------------------------------- YZ emniyet zincirinde değil
@pytest.mark.parametrize("eylem", TUM_EYLEMLER)
@pytest.mark.parametrize("durum", [lab1.CALISIYOR, lab1.DURUYOR, lab1.GUVENLI_DURUM])
def test_yz_hicbir_ciktiyla_acil_stopu_etkilemez(durum, eylem):
    assert m.hat_cevrimi(durum, lab1.girisler(acil_stop=True), eylem) == lab1.GUVENLI_DURUM


@pytest.mark.parametrize("eylem", TUM_EYLEMLER)
def test_yz_guvenli_durumdan_makineyi_calistiramaz(eylem):
    assert m.hat_cevrimi(lab1.GUVENLI_DURUM, lab1.girisler(start=True), eylem) == lab1.GUVENLI_DURUM


@pytest.mark.parametrize("istek", m.d("lab7.hiz_istegi_taramasi"))
def test_yz_hiz_istegi_normal_kumandada_kesilir(istek):
    assert m.normal_kumanda_hizi(istek) <= m.d("lab7.hiz_istegi_ust_siniri")


@pytest.mark.parametrize("istek", m.d("lab7.hiz_istegi_taramasi"))
def test_kumanda_atlatilsa_da_hiz_izleme_eli_korur(istek):
    assert not lab1.el_besleme_agzina_ulasir_mi(m.acil_stop_mesafesi(istek, kumanda_kesiyor=False))


@pytest.mark.xfail(strict=True, reason="emniyet tarafında hız izleme yoksa YZ hız isteği sınırı aşar (beklenen)")
```

```python
def test_hiz_izleme_olmadan_da_eli_korur():
    for istek in m.d("lab7.hiz_istegi_taramasi"):
        assert not lab1.el_besleme_agzina_ulasir_mi(
            m.acil_stop_mesafesi(istek, kumanda_kesiyor=False, hiz_izleme=False))


# ---------------------------------------------------------------- 7.4 Ortak neden
def test_bagimsiz_hatalarda_ikinci_kanal_kacak_hedefini_saglar():
    r = m.metrikler(m.veri_uret(), m.esik_sec(md=True), md=True, ortak_neden=0.0)
    assert r["kacak"] <= m.d("lab7.hedef_kacak_orani")


def test_ortak_neden_payi_artinca_kacak_artar():
    kacaklar = [r["kacak"] for _, r in m.ortak_neden_taramasi()]
    assert kacaklar == sorted(kacaklar) and kacaklar[-1] > kacaklar[0]


def test_ortak_neden_esigi_asilinca_kacak_hedefi_asilir():
    """V20'deki bağımsızlık varsayımı kritik: küçük bir ortak neden payı hedefi bozar."""
    rho = m.ortak_neden_esigi()
    veri, e, hedef = m.veri_uret(), m.esik_sec(md=True), m.d("lab7.hedef_kacak_orani")
    assert m.metrikler(veri, e, md=True, ortak_neden=rho * 0.99)["kacak"] <= hedef
    assert m.metrikler(veri, e, md=True, ortak_neden=rho * 1.01)["kacak"] > hedef


# ---------------------------------------------------------------- 7.5 Kayma ve izleme
def test_kaymasiz_veri_devreye_almadaki_kumeyle_aynidir():
    assert m.veri_uret(0.0) == m.veri_uret()


def test_kusurlu_skorlar_saglama_kaydikca_kacak_hedefi_asilir():
    assert m.izleme_sonucu()["hedef_asim"] is not None


def test_operator_yuku_kaymayi_gostermez():
    """Kaçak toleransı aşıldığında bile operatöre sorulan oran alarm eşiğine ulaşmıyor."""
    iz = m.izleme_sonucu()
    assert iz["tolerans_asim"] is not None
    assert iz["alarm_sor"] is None or iz["alarm_sor"] > iz["tolerans_asim"]


def test_celiski_alarmi_kacak_toleransi_asilmadan_once_calar():
    """A7: YZ ile bağımsız dedektör arasındaki çelişki oranı izlenir; kayma tolerans içinde yakalanır."""
    iz = m.izleme_sonucu()
    assert iz["alarm_celiski"] is not None and iz["alarm_celiski"] < iz["tolerans_asim"]


@pytest.mark.xfail(strict=True, reason="Kaçak, devreye almada hedefe çok yakın; hiçbir izleme hedef aşılmadan önce davranamıyor")
def test_celiski_alarmi_kacak_hedefi_asilmadan_once_calar():
    iz = m.izleme_sonucu()
    assert iz["alarm_celiski"] < iz["hedef_asim"]
```

`params.yaml` — LAB 7 bölümü:

efo_sim/out/lab7_params.yaml

```yaml
lab7:
  ornek_sayisi: {deger: 20000, birim: "adet", etiket: "VARSAYIM", kaynak: "V20", not: "Sentetik değerlendirme kümesi"}
  tohum: {deger: 42, birim: "-", etiket: "VARSAYIM", kaynak: "-", not: "Tekrarlanabilirlik için rastgele sayı tohumu"}
  kusur_orani:
    {deger: 0.02, birim: "oran", etiket: "VARSAYIM", kaynak: "V20",
     not: "Bantta yabancı cisim (metal, taş) ya da aşırı boy parça oranı"}
  metal_payi:
    {deger: 0.6, birim: "oran", etiket: "VARSAYIM", kaynak: "V20", not: "Kusurların metal olan kısmı"}
  skor_kusurlu:
    {deger: [6.0, 2.0], birim: "Beta(a, b)", etiket: "VARSAYIM", kaynak: "V20",
     not: "Kusurlu parçada modelin verdiği p(kusur) dağılımı"}
  skor_saglam:
    {deger: [1.5, 9.0], birim: "Beta(a, b)", etiket: "VARSAYIM", kaynak: "V20",
     not: "Sağlam parçada modelin verdiği p(kusur) dağılımı"}
  karar_siniri:
    {deger: 0.5, birim: "p", etiket: "VARSAYIM", kaynak: "V20",
     not: "p(kusur) ≥ 0,5 → RED; altı → OK (ikili sınıflandırıcının olağan sınırı)"}
  hedef_kacak_orani:
    {deger: 0.01, birim: "oran", etiket: "VARSAYIM", kaynak: "V20",
```

```
      not: "Kusurların en fazla %1'i otomatik OK alabilir"}
  insan_kapasitesi:
    {deger: 0.05, birim: "oran", etiket: "VARSAYIM", kaynak: "V20",
     not: "Operatörün onaylayabileceği en yüksek parça oranı"}
  esik_araligi:
    {deger: [0.5, 0.99], birim: "güven", etiket: "VARSAYIM", kaynak: "-", not: "Güven eşiği taraması"}
  esik_adim_sayisi: {deger: 50, birim: "adet", etiket: "VARSAYIM", kaynak: "-", not: "Tarama çözünürlüğü"}
  md_kacirma_orani:
    {deger: 0.05, birim: "oran", etiket: "VARSAYIM", kaynak: "V20",
     not: "YZ'den bağımsız metal dedektörünün metal parçayı kaçırma oranı"}
  girdi_kontrolleri:
    deger: {azami_kare_yasi_ms: 200, donuk_kare_sayisi: 5, parlaklik_araligi: [0.08, 0.92], asgari_keskinlik:
0.15}
    etiket: "VARSAYIM"
    kaynak: "V20"
    not: "Kare yaşı, aynı karenin tekrarı, parlaklık ve keskinlik sınırları"
  hiz_istegi_ust_siniri:
    {deger: 1.0, birim: "× nominal", etiket: "VARSAYIM", kaynak: "V20",
     not: "Normal kumanda YZ'nin dozaj hızı isteğini nominalde keser"}
  hiz_istegi_taramasi:
    {deger: [0.5, 1.0, 1.5, 2.0, 3.0], birim: "× nominal", etiket: "VARSAYIM", kaynak: "-",
     not: "Test edilen YZ hız istekleri"}
  # 7.4 Ortak neden: YZ ve metal dedektörü hatalarının bağımsızlığı (V20) gevşetilir
  ortak_neden_taramasi:
    {deger: [0.0, 0.2, 21], birim: "[alt, üst, adım sayısı]", etiket: "VARSAYIM", kaynak: "V26",
     not: "Ortak neden payı ρ: YZ'nin kaçırdığı metal parçayı dedektörün de aynı nedenle kaçırma olasılığı"}
  # 7.5 Kayma: kusurlu parçaların skor dağılımı sağlam parçalarınkine doğru kayar (yeni yabancı cisim türü)
  kayma_taramasi:
    {deger: [0.0, 0.3, 13], birim: "[alt, üst, adım sayısı]", etiket: "VARSAYIM", kaynak: "V26",
     not: "δ = 0 devreye alma; δ = 1 kusurlu skorlar sağlamlarla aynı dağılımda"}
  izleme_alarm_carpani:
    {deger: 1.5, birim: "× devreye alma değeri", etiket: "VARSAYIM", kaynak: "V26, Y29",
     not: "İzlenen oran devreye almadaki değerinin bu katına çıkınca alarm"}
  kayma_tolerans_carpani:
    {deger: 2.0, birim: "× hedef kaçak", etiket: "VARSAYIM", kaynak: "V26",
     not: "Alarm, kaçak oranı hedefin bu katına çıkmadan çalmalı (A7)"}
```

pytest çıktısı:

efo_sim/out/pytest_lab7.txt

```
============================= test session starts ==============================
platform linux -- Python 3.14.8, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python
rootdir: /home/ydogan/Masaüstü/safety40-efo-konveyor
configfile: pytest.ini
plugins: asyncio-1.3.0, anyio-4.13.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function
collecting ... collected 49 items

efo_sim/tests/test_lab7_ai.py::test_esiksiz_model_kacak_hedefini_karsilamaz PASSED [  2%]
efo_sim/tests/test_lab7_ai.py::test_secilen_esik_kacak_hedefini_operator_kapasitesi_icinde_karsilar PASSED [  4%]
efo_sim/tests/test_lab7_ai.py::test_bagimsiz_metal_dedektoru_operator_yukunu_azaltir PASSED [  6%]
efo_sim/tests/test_lab7_ai.py::test_esik_arttikca_kacak_azalir_operator_yuku_artar PASSED [  8%]
efo_sim/tests/test_lab7_ai.py::test_red_guven_dusuk_olsa_da_ayirir PASSED [ 10%]
efo_sim/tests/test_lab7_ai.py::test_normal_girdi_supheli_degil PASSED    [ 12%]
efo_sim/tests/test_lab7_ai.py::test_supheli_girdide_besleme_durur[kare0-gecmis0-kare eski] PASSED [ 14%]
efo_sim/tests/test_lab7_ai.py::test_supheli_girdide_besleme_durur[kare1-gecmis1-donmu\u015f kare] PASSED [ 16%]
efo_sim/tests/test_lab7_ai.py::test_supheli_girdide_besleme_durur[kare2-gecmis2-parlakl\u0131k s\u0131n\u0131r
d\u0131\u015f\u0131] PASSED [ 18%]
efo_sim/tests/test_lab7_ai.py::test_supheli_girdide_besleme_durur[kare3-gecmis3-bulan\u0131k / kapal\u0131 lens]
PASSED [ 20%]
efo_sim/tests/test_lab7_ai.py::test_metal_dedektoru_yz_ile_celisirse_parca_ayrilir PASSED [ 22%]
efo_sim/tests/test_lab7_ai.py::test_esigin_altinda_ok_otomatik_verilmez[0.4] PASSED [ 24%]
efo_sim/tests/test_lab7_ai.py::test_esigin_altinda_ok_otomatik_verilmez[0.45] PASSED [ 26%]
efo_sim/tests/test_lab7_ai.py::test_esigin_altinda_ok_otomatik_verilmez[0.49] PASSED [ 28%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[\xc7ALI\u015eIYOR-GE\xc7] PASSED [ 30%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[\xc7ALI\u015eIYOR-AYIR] PASSED [ 32%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[\xc7ALI\u015eIYOR-OPERAT\xd6RE SOR]
PASSED [ 34%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[\xc7ALI\u015eIYOR-BESLEMEY\u0130 DURDUR]
PASSED [ 36%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[DURUYOR-GE\xc7] PASSED [ 38%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[DURUYOR-AYIR] PASSED [ 40%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[DURUYOR-OPERAT\xd6RE SOR] PASSED [ 42%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[DURUYOR-BESLEMEY\u0130 DURDUR] PASSED
[ 44%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[G\xdcVENL\u0130_DURUM-GE\xc7] PASSED
```

```
[ 46%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[G\xdcVENL\u0130_DURUM-AYIR] PASSED [ 48%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[G\xdcVENL\u0130_DURUM-OPERAT\xd6RE SOR]
PASSED [ 51%]
efo_sim/tests/test_lab7_ai.py::test_yz_hicbir_ciktiyla_acil_stopu_etkilemez[G\xdcVENL\u0130_DURUM-BESLEMEY\u0130
DURDUR] PASSED [ 53%]
efo_sim/tests/test_lab7_ai.py::test_yz_guvenli_durumdan_makineyi_calistiramaz[GE\xc7] PASSED [ 55%]
efo_sim/tests/test_lab7_ai.py::test_yz_guvenli_durumdan_makineyi_calistiramaz[AYIR] PASSED [ 57%]
efo_sim/tests/test_lab7_ai.py::test_yz_guvenli_durumdan_makineyi_calistiramaz[OPERAT\xd6RE SOR] PASSED [ 59%]
efo_sim/tests/test_lab7_ai.py::test_yz_guvenli_durumdan_makineyi_calistiramaz[BESLEMEY\u0130 DURDUR] PASSED [ 61%]
efo_sim/tests/test_lab7_ai.py::test_yz_hiz_istegi_normal_kumandada_kesilir[0.5] PASSED [ 63%]
efo_sim/tests/test_lab7_ai.py::test_yz_hiz_istegi_normal_kumandada_kesilir[1.0] PASSED [ 65%]
efo_sim/tests/test_lab7_ai.py::test_yz_hiz_istegi_normal_kumandada_kesilir[1.5] PASSED [ 67%]
efo_sim/tests/test_lab7_ai.py::test_yz_hiz_istegi_normal_kumandada_kesilir[2.0] PASSED [ 69%]
efo_sim/tests/test_lab7_ai.py::test_yz_hiz_istegi_normal_kumandada_kesilir[3.0] PASSED [ 71%]
efo_sim/tests/test_lab7_ai.py::test_kumanda_atlatilsa_da_hiz_izleme_eli_korur[0.5] PASSED [ 73%]
efo_sim/tests/test_lab7_ai.py::test_kumanda_atlatilsa_da_hiz_izleme_eli_korur[1.0] PASSED [ 75%]
efo_sim/tests/test_lab7_ai.py::test_kumanda_atlatilsa_da_hiz_izleme_eli_korur[1.5] PASSED [ 77%]
efo_sim/tests/test_lab7_ai.py::test_kumanda_atlatilsa_da_hiz_izleme_eli_korur[2.0] PASSED [ 79%]
efo_sim/tests/test_lab7_ai.py::test_kumanda_atlatilsa_da_hiz_izleme_eli_korur[3.0] PASSED [ 81%]
efo_sim/tests/test_lab7_ai.py::test_hiz_izleme_olmadan_da_eli_korur XFAIL [ 83%]
efo_sim/tests/test_lab7_ai.py::test_bagimsiz_hatalarda_ikinci_kanal_kacak_hedefini_saglar PASSED [ 85%]
efo_sim/tests/test_lab7_ai.py::test_ortak_neden_payi_artinca_kacak_artar PASSED [ 87%]
efo_sim/tests/test_lab7_ai.py::test_ortak_neden_esigi_asilinca_kacak_hedefi_asilir PASSED [ 89%]
efo_sim/tests/test_lab7_ai.py::test_kaymasiz_veri_devreye_almadaki_kumeyle_aynidir PASSED [ 91%]
efo_sim/tests/test_lab7_ai.py::test_kusurlu_skorlar_saglama_kaydikca_kacak_hedefi_asilir PASSED [ 93%]
efo_sim/tests/test_lab7_ai.py::test_operator_yuku_kaymayi_gostermez PASSED [ 95%]
efo_sim/tests/test_lab7_ai.py::test_celiski_alarmi_kacak_toleransi_asilmadan_once_calar PASSED [ 97%]
efo_sim/tests/test_lab7_ai.py::test_celiski_alarmi_kacak_hedefi_asilmadan_once_calar XFAIL [100%]

======================== 47 passed, 2 xfailed in 5.54s ========================
```

Kaynaklar

R = derin araştırma raporundaki kaynak numarası; Y = bu çalışmada eklenen kaynak.

- **Y27** Regulation (EU) 2023/1230 (Machinery Regulation) — Annex I Part A, Annex III 1.1.9 ve 1.2.1 — https://eur-lex.europa.eu/eli/reg/2023/1230/oj
- **Y28** Regulation (EU) 2024/1689 (AI Act) — Article 6, Annex I — https://eur-lex.europa.eu/eli/reg/2024/1689/oj
- **Y29** ISO/IEC TR 5469:2024 — Artificial intelligence — Functional safety and AI systems — https://www.iso.org/standard/81283.html